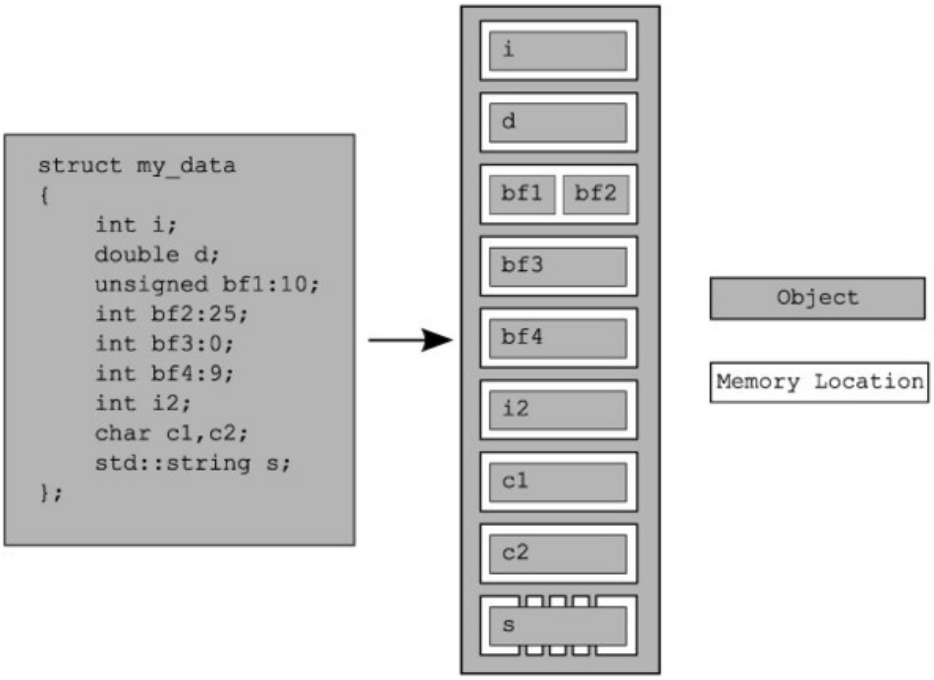
5.1.1. Objects and memory locations

All data in a C++ program is made up of *objects*. This is not to say that you can create a new class derived from `int`, or that the fundamental types have member functions, or any of the other consequences often implied when people say "everything is an object" when discussing a language like Smalltalk or Ruby. It's just a statement about the building blocks of data in C++. The C++ Standard defines an object as "a region of storage," although it goes on to assign properties to these objects, such as their type and lifetime.

Some of these objects are simple values of a fundamental type such as `int` or `float`, whereas others are instances of user-defined classes. Some objects (such as arrays, instances of derived classes, and instances of classes with non-`static` data members) have subobjects, but others don't.

Whatever its type, an object is stored in one or more *memory locations*. Each such memory location is either an object (or subobject) of a scalar type such as `unsigne dshort` or `my_class*` or a sequence of adjacent bit fields. If you use bit fields, this is an important point to note: though adjacent bit fields are distinct objects, they're still counted as the same memory location. Figure 5.1 shows how a `struct` divides into objects and memory locations.

Figure 5.1. The division of a struct into objects and memory locations



First, the entire `struct` is one object, which consists of several subobjects, one for each data member. The bit fields `bf1` and `bf2` share a memory location, and the `std::string` object `s` consists of several memory locations internally, but otherwise each member has its own memory location. Note how the zero-length bit field `bf3` separates `bf4` into its own memory location.